

Εργασία -- Συγκριτική μελέτη αλγορίθμων ταξινόμησης

Μέλη Ομάδας για την Εργασία στο μάθημα Αλγόριθμοι και Πολυπλοκότητα:

Νικόλαος Κατσικαβάλης, AM:02618, email: nkatsikavalis@uth.gr

Γεώργιος Παπαμικρούλης, AM:02675, email: gparamikr@uth.gr

Μέρος Α: Θεωρητική μελέτη

• Δραστηριότητα Α.1

Bubble Sort:

1. για $i = 1$ έως $A.\text{μήκος} - 1$
2. έγιναν_ανταλλαγές = Ψευδής
3. για $j = A.\text{μήκος}$ αντίστροφα έως $i + 1$
4. αν $A[j] < A[j-1]$
5. εναλλάσσουμε τα $A[j]$ και $A[j-1]$
6. έγιναν_ανταλλαγές = Αληθής
7. αν έγιναν_ανταλλαγές == Ψευδής, τότε Σταμάτα

Insert sort:

1. Για $j = 2$ έως $A.\text{μήκος}$
2. κλειδί = $A[j]$
3. //Ενθέτουμε το $A[j]$ ταξινομημένη ακολουθία $A[1..j-1]$.
4. $i = j - 1$
5. Ενόσω $i > 0$ και $A[i] > \text{κλειδί}$
6. $A[i+1] = A[i]$
7. $i = i - 1$
8. $A[i+1] = \text{κλειδί}$

Merge sort:

Αναδρομικός Μηχανισμός:

1. Αν $p < r$:
2. $q = \lfloor (p + r) / 2 \rfloor$
3. Merge-Sort(A, p, q)
4. Merge-Sort($A, q + 1, r$)

5. Merge(A, p, q, r)

Διαδικασία Συγχώνευσης (Merge):

1. $N1 = q - p + 1$
2. $N2 = r - q$
3. Έστω $L[1..N1+1]$ και $R[1..N2+1]$
4. Για $i=1$ έως $N1$
5. $L[i] = A[p+i-1]$
6. Για $j=1$ έως $N2$
7. $R[j] = A[q+j]$
8. $L[N1+1] = \text{άπειρο}$
9. $R[N2+1] = \text{άπειρο}$
10. $i=1$
11. $j=1$
12. Για $k=p$ έως r
13. Αν $L[i] \leq R[j]$
14. $A[k] = L[i]$
15. $i=i+1$
16. Άλλως $A[k] = R[j]$
17. $j=j+1$

- Δραστηριότητα A.2

Bubble Sort:

Best Case: $O(n)$. Συμβαίνει όταν ο πίνακας είναι ήδη ταξινομημένος. Χάρη στον έλεγχο "έγιναν_ανταλλαγές", ο αλγόριθμος τερματίζει μετά το πρώτο πέρασμα αν δεν γίνει καμία εναλλαγή στοιχείων.

Average Case: $O(n^2)$. Για μια τυχαία διάταξη δεδομένων, απαιτούνται περίπου $n^2/2$ συγκρίσεις.

Worst Case: $O(n^2)$. Συμβαίνει όταν ο πίνακας είναι αντίστροφα ταξινομημένος.

Insertion Sort:

Best Case: $O(n)$. Συμβαίνει όταν ο πίνακας είναι ήδη ταξινομημένος. Ο εσωτερικός βρόχος "Ενόσω" ελέγχει μόνο μία φορά και σταματά αμέσως.

Average Case: $O(n^2)$. Για τυχαία δεδομένα, κάθε νέο στοιχείο συγκρίνεται με τα μισά από τα προηγούμενα στοιχεία κατά μέσο όρο.

Worst Case: $O(n^2)$. Συμβαίνει όταν ο πίνακας είναι αντίστροφα ταξινομημένος, καθώς κάθε στοιχείο πρέπει να μετακινηθεί στην αρχή της λίστας.

Merge Sort:

Best Case: $O(n \log n)$. Ο πίνακας διαιρείται πάντα στο μισό ($\log n$ επίπεδα) και η συγχώνευση (merge) απαιτεί n βήματα σε κάθε επίπεδο.

Average Case): $O(n \log n)$.

Worst Case): $O(n \log n)$. Ακόμα και στην πλήρη αποδιοργάνωση, ο αριθμός των διαιρέσεων και των συγχωνεύσεων παραμένει ο ίδιος.

- Δραστηριότητα A.3

Αλγόριθμος	Best Case	Average Case	Worst Case
BubbleSort	$O(n)$	$O(n^2)$	$O(n^2)$
InsertionSort	$O(n)$	$O(n^2)$	$O(n^2)$

MergeSort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
-----------	---------------	---------------	---------------

- **Δραστηριότητα A.4**

Ο αλγόριθμος που επηρεάζεται περισσότερο από το αν τα δεδομένα μας είναι ήδη "σχεδόν ταξινομημένα" είναι ο Insertion Sort.

Η λογική του Insertion Sort είναι να παίρνει ένα στοιχείο και να το τοποθετεί στη σωστή θέση ανάμεσα στα προηγούμενα. Αν ο πίνακας είναι ήδη ταξινομημένος, ο αλγόριθμος απλώς κοιτάζει το κάθε στοιχείο, βλέπει ότι είναι μεγαλύτερο από το προηγούμενο και προχωράει αμέσως στο επόμενο χωρίς να κάνει καμία μετακίνηση. Έτσι, από εκεί που κανονικά χρειάζεται $O(n^2)$ χρόνο, στην περίπτωση αυτή τελειώνει πολύ γρήγορα σε $O(n)$.

Σε σχέση με τους άλλους δύο:

- Ο Merge Sort είναι "σταθερός" και δεν τον νοιάζει η αρχική σειρά. Θα κάνει πάντα τον ίδιο αριθμό διαιρέσεων και συγχωνεύσεων ($O(n \log n)$), είτε ο πίνακας είναι ανακατεμένος είτε τέλειος.
- Ο Bubble Sort, στην βελτιστοποιημένη έκδοση με τη χρήση της μεταβλητής *swapped*, αντιλαμβάνεται την ήδη ταξινομημένη είσοδο. Αν σε ένα πλήρες πέρασμα δεν σημειωθεί καμία εναλλαγή, ο αλγόριθμος τερματίζει πρόωρα, επιτυγχάνοντας χρόνο $O(n)$, όπως επιβεβαιώνεται και από τις μετρήσεις μας στο Μέρος Γ.
Συμπέρασμα: Ο Insertion Sort «εκμεταλλεύεται» την αρχική διάταξη. Όσο πιο ταξινομημένη είναι η είσοδος (δηλαδή όσο λιγότερες αντιστροφές έχει), τόσο λιγότερες συγκρίσεις και μετακινήσεις κάνει, άρα τρέχει πολύ πιο γρήγορα

Μέρος Β: Μέτρα αρχικής ταξινόμησης

- **Δραστηριότητα B.1**

Ως descent ορίζουμε κάθε περίπτωση όπου ένα στοιχείο είναι μεγαλύτερο από το επόμενο του, δηλαδή $A[i] > A[i+1]$.

Παραδείγματα ακολουθιών μήκους 8:

1) 0 descents: [1, 2, 3, 4, 5, 6, 7, 8]

Η ακολουθία είναι πλήρως ταξινομημένη, οπότε δεν υπάρχει κανένα σημείο όπου ένα στοιχείο να είναι μεγαλύτερο από το επόμενο.

2) 2 descents: [1, 2, 4, 3, 5, 6, 8, 7]

Εδώ έχουμε descents στα σημεία (4, 3) και (8, 7). Σε όλες τις άλλες θέσεις, το επόμενο στοιχείο είναι μεγαλύτερο.

3) 7 descents: [8, 7, 6, 5, 4, 3, 2, 1]

Η ακολουθία είναι αντίστροφα ταξινομημένη. Σε κάθε βήμα (από το 8 στο 7, από το 7 στο 6 κ.ο.κ.), το τρέχον στοιχείο είναι μεγαλύτερο από το επόμενο. Σε μια λίστα μήκους $n=8$, ο μέγιστος αριθμός descents είναι $n-1 = 7$.

• Δραστηριότητα B.2

1. Ακολουθία [1, 2, 3, 4, 5] :

- Εδώ κάθε στοιχείο είναι μικρότερο από όλα τα επόμενά του.
- Δεν υπάρχει κανένα ζεύγος που να ικανοποιεί τη συνθήκη $A[i] > A[j]$.
- Αριθμός Αντιστροφών: 0.

2. Ακολουθία [2, 1, 3, 5, 4]

Ελέγχουμε τα ζεύγη:

- Για το 2: Είναι μεγαλύτερο από το 1. (1 αντιστροφή: ζεύγος (2,1)).
- Για το 1: Δεν είναι μεγαλύτερο από κανένα επόμενο.
- Για το 3: Δεν είναι μεγαλύτερο από κανένα επόμενο.
- Για το 5: Είναι μεγαλύτερο από το 4. (1 αντιστροφή: ζεύγος (5,4)).
- Συνολικός αριθμός αντιστροφών: $1 + 1 = 2$.

3. Ακολουθία [5, 4, 3, 2, 1]

Εδώ κάθε στοιχείο είναι μεγαλύτερο από όλα όσα ακολουθούν:

- Για το 5: Μεγαλύτερο από τα 4, 3, 2, 1 (4 αντιστροφές).
- Για το 4: Μεγαλύτερο από τα 3, 2, 1 (3 αντιστροφές).
- Για το 3: Μεγαλύτερο από τα 2, 1 (2 αντιστροφές).
- Για το 2: Μεγαλύτερο από το 1 (1 αντιστροφή).
- Για το 1: Κανένα επόμενο.
- Συνολικός αριθμός αντιστροφών: $4 + 3 + 2 + 1 = 10$.

• Δραστηριότητα B.3

```
def count_descents(A):
    # Μέτρηση περιπτώσεων όπου  $A[i] > A[i+1]$ 
    count = 0
    for i in range(len(A) - 1):
        if A[i] > A[i+1]:
            count += 1
    return count

def count_inversions(A):
    # Μέτρηση ζευγών (i, j) με  $i < j$  και  $A[i] > A[j]$ 
    count = 0
    n = len(A)
    for i in range(n):
        for j in range(i + 1, n):
            if A[i] > A[j]:
                count += 1
    return count

# Δοκιμή με τη λίστα της άσκησης
if __name__ == "__main__":
    test_list = [2, 1, 3, 5, 4]
    print(f"Δεδομένα εισόδου: {test_list}")
    print(f"Πλήθος Descents: {count_descents(test_list)}")
    print(f"Πλήθος Αντιστροφών: {count_inversions(test_list)}")
```

TERMINAL:

Δεδομένα εισόδου: [2, 1, 3, 5, 4]
Πλήθος Descents: 2

Πλήθος Αντιστροφών: 2

Όπως βλέπουμε στο παραπάνω αποτέλεσμα, η συνάρτηση υπολογίζει σωστά τις τιμές για τη δοκιμαστική λίστα [2, 1, 3, 5, 4]. Τα αποτελέσματα (2 descents και 2 αντιστροφές) ταυτίζονται απόλυτα με τους χειροκίνητους υπολογισμούς που πραγματοποιήσαμε στη Δραστηριότητα B.2

- **Δραστηριότητα B.4**

```
def create_descents(n, k):
    # Δημιουργία λίστας n στοιχείων με k descents
    if k < 0 or k >= n:
        return "Error: k must be between 0 and n-1"
    # Αρχική αύξουσα λίστα
    data = list(range(1, n + 1))
    # Αντιστροφή τμήματος για δημιουργία descents
    data[k+1] = reversed(data[k+1])
    return data

def create_inversions(n, k):
    # Δημιουργία λίστας n στοιχείων με k αντιστροφές
    limit = n * (n - 1) // 2
    if k < 0 or k > limit:
        return f"Error: k must be between 0 and {limit}"
    source = list(range(1, n + 1))
    result = []
    for i in range(n):
        # Υπολογισμός διαθέσιμων θέσεων στα δεξιά
        slots = n - 1 - i
        if k >= slots:
            # Επιλογή μεγαλύτερου στοιχείου
            val = source.pop()
            result.append(val)
            k -= slots
        else:
            # Επιλογή στοιχείου για ακριβή συμπλήρωση του k
            val = source.pop(k)
            result.append(val)
            k = 0
    return result

# Παραδείγματα εκτέλεσης
if __name__ == "__main__":
    n, k_des, k_inv = 8, 3, 12
```

```
print(f"Λίστα με {k_des} descents: {create_descents(n, k_des)}")
print(f"Λίστα με {k_inv} inversions: {create_inversions(n, k_inv)}")
```

TERMINAL:

Λίστα με 3 descents: [4, 3, 2, 1, 5, 6, 7, 8]

Λίστα με 12 inversions: [8, 6, 1, 2, 3, 4, 5, 7]

Το τερματικό επιβεβαιώνει την ορθή λειτουργία των συναρτήσεων κατασκευής. Οι παραχθείσες λίστες των 8 στοιχείων διαθέτουν ακριβώς τον ζητούμενο αριθμό από descents (3) και αντιστροφές (12) αντίστοιχα, επαληθεύοντας τη λογική σχεδιασμού μας για την παραγωγή δεδομένων με ελεγχόμενη αταξία.

Σκεπτικό κατασκευής της λίστας με k αντιστροφές:

- 1) Προετοιμασία: Ξεκινάμε με όλους τους αριθμούς από το 1 έως το n διαθέσιμους σε μια ταξινομημένη βοηθητική λίστα.
- 2) Βήμα-Βήμα Επιλογή: Σε κάθε θέση της νέας μας ακολουθίας, ελέγχουμε πόσες αντιστροφές μας απομένουν για να φτάσουμε το k :
 - Όταν το k είναι μεγάλο: Αν οι αντιστροφές που χρειαζόμαστε είναι περισσότερες από τις θέσεις που μένουν ελεύθερες στα δεξιά, τότε παίρνουμε τον μεγαλύτερο διαθέσιμο αριθμό. Αυτός, επειδή μπαίνει μπροστά από όλους τους άλλους, δημιουργεί αυτόματα τον μέγιστο δυνατό αριθμό αντιστροφών για εκείνη τη θέση.
 - Όταν το k μικρύνει: Αν οι αντιστροφές που απομένουν είναι λιγότερες από τις διαθέσιμες θέσεις, διαλέγουμε από τη βοηθητική μας λίστα ακριβώς εκείνο το στοιχείο που έχει από κάτω του τόσα νούμερα όσα μας λείπουν για να συμπληρώσουμε το k .
- 3) Ολοκλήρωση: Μόλις το k μηδενιστεί, δεν θέλουμε να προσθέσουμε άλλες αντιστροφές. Έτσι, παίρνουμε όλους τους αριθμούς που περίσσεψαν και τους βάζουμε στη σειρά, από τον μικρότερο στον μεγαλύτερο, μέχρι να γεμίσει η λίστα μας.

Μέρος Γ: Πειραματική μελέτη

- Δραστηριότητα Γ.1 & Γ.2

```
import time
import sys
import random
```

```
sys.setrecursionlimit(10000)
```

```
def bubble_sort(arr):
```

```
    # Δραστηριότητα Γ.2: Μετρητές συγκρίσεων και χρόνου
```

```
    comparisons = 0
```

```
    n = len(arr)
```

```
    start_time = time.perf_counter()
```

```
    for i in range(n - 1):
```

```
        swapped = False # Προσθήκη για αρμονία με τη θεωρία (Α.1)
```

```
        for j in range(n - 1, i, -1):
```

```
            comparisons += 1
```

```
            if arr[j] < arr[j-1]:
```

```
                arr[j], arr[j-1] = arr[j-1], arr[j]
```

```
                swapped = True
```

```
        if not swapped:
```

```
            break # Σταματάει αν ο πίνακας είναι ήδη ταξινομημένος
```

```
    end_time = time.perf_counter()
```

```
    duration = end_time - start_time
```

```
    return comparisons, duration
```

```
def insertion_sort(arr):
```

```
    # Δραστηριότητα Γ.2: Μετρητές συγκρίσεων και χρόνου
```

```
    comparisons = 0
```

```
    n = len(arr)
```

```
    start_time = time.perf_counter()
```

```
    for j in range(1, n):
```

```
        key = arr[j]
```

```
        i = j - 1
```

```
        while i >= 0:
```

```
            comparisons += 1
```

```

    if arr[i] > key:
        arr[i + 1] = arr[i]
        i -= 1
    else:
        break
    arr[i + 1] = key

```

```

end_time = time.perf_counter()
duration = end_time - start_time
return comparisons, duration

```

```

def merge_sort_recursive(arr, p, r, stats):
    if p < r:
        q = (p + r) // 2
        merge_sort_recursive(arr, p, q, stats)
        merge_sort_recursive(arr, q + 1, r, stats)
        merge(arr, p, q, r, stats)

```

```

def merge(arr, p, q, r, stats):
    n1 = q - p + 1
    n2 = r - q
    L = arr[p:p + n1]
    R = arr[q + 1:q + 1 + n2]

```

```

    L.append(float('inf'))
    R.append(float('inf'))

```

```

    i = 0
    j = 0
    for k in range(p, r + 1):
        stats['comps'] += 1
        if L[i] <= R[j]:
            arr[k] = L[i]
            i += 1
        else:
            arr[k] = R[j]
            j += 1

```

```

def merge_sort(arr):
    # Δραστηριότητα Γ.2: Μετρητές συγκρίσεων και χρόνου
    stats = {'comps': 0}

```

```

start_time = time.perf_counter()

merge_sort_recursive(arr, 0, len(arr) - 1, stats)

end_time = time.perf_counter()
duration = end_time - start_time
return stats['comps'], duration

if __name__ == "__main__":
    n = 1000
    test_data = [random.randint(1, 10000) for _ in range(n)]

    print(f"--- Πειραματική Μελέτη (n={n}) ---")

    c_b, t_b = bubble_sort(test_data.copy())
    print(f"Bubble Sort -> Συγκρίσεις: {c_b}, Χρόνος: {t_b:.6f}s")

    c_i, t_i = insertion_sort(test_data.copy())
    print(f"Insertion Sort -> Συγκρίσεις: {c_i}, Χρόνος: {t_i:.6f}s")

    c_m, t_m = merge_sort(test_data.copy())
    print(f"Merge Sort -> Συγκρίσεις: {c_m}, Χρόνος: {t_m:.6f}s")

```

TERMINAL:

```
--- Πειραματική Μελέτη (n=1000) ---
```

```
Bubble Sort -> Συγκρίσεις: 498465, Χρόνος: 0.175815s
```

```
Insertion Sort -> Συγκρίσεις: 256415, Χρόνος: 0.052686s
```

```
Merge Sort -> Συγκρίσεις: 9976, Χρόνος: 0.004880s
```

Σε αυτό το πρώτο πείραμα (n=1000) διαφαίνεται ήδη η τεράστια διαφορά στην αποδοτικότητα των αλγορίθμων. Ο Merge Sort απαιτεί σχεδόν 50 φορές λιγότερες συγκρίσεις από τον Bubble Sort, ολοκληρώνοντας την ταξινόμηση σε κλάσματα του δευτερολέπτου. Παράλληλα, ο Insertion Sort αποδεικνύεται περίπου 3 φορές ταχύτερος από τον Bubble Sort στα τυχαία δεδομένα, παρότι μοιράζονται την ίδια θεωρητική πολυπλοκότητα $O(n^2)$.

- **Δραστηριότητα Γ.3**

```
import random
```

```
def create_descents(n, k):
```

```

data = list(range(1, n + 1))
data[:k+1] = reversed(data[k+1:])
return data

def generate_all_datasets(n):
    datasets = {}

    # 1. Ήδη ταξινομημένη ακολουθία
    datasets["Sorted"] = list(range(1, n + 1))

    # 2. Αντίστροφα ταξινομημένη ακολουθία
    datasets["Reverse"] = list(range(n, 0, -1))

    # 3. Τυχαία ακολουθία
    datasets["Random"] = random.sample(range(1, n * 10), n)

    # 4. Ακολουθία με μικρό αριθμό descents
    k_small = max(1, int(n * 0.1))
    datasets["Few_Descents"] = create_descents(n, k_small)

    # 5. Ακολουθία με μεγάλο αριθμό descents
    k_large = int(n * 0.8)
    datasets["Many_Descents"] = create_descents(n, k_large)

    return datasets

if __name__ == "__main__":
    n_test = 10
    data = generate_all_datasets(n_test)
    for name, s in data.items():
        print(f"{name:15}: {s}")

```

TERMINAL:

```

Sorted      : [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Reverse     : [10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
Random      : [41, 63, 50, 92, 7, 10, 13, 62, 67, 97]
Few_Descents : [2, 1, 3, 4, 5, 6, 7, 8, 9, 10]
Many_Descents : [9, 8, 7, 6, 5, 4, 3, 2, 1, 10]

```

Το μικρό δείγμα των 10 στοιχείων επιβεβαιώνει ότι οι γεννήτριες παράγουν σωστά τις πέντε διαφορετικές διατάξεις δεδομένων (Sorted, Reverse, Random, Few Descents, Many Descents). Η σωστή παραγωγή

αυτών των συνόλων είναι κρίσιμη, καθώς εξασφαλίζει την εγκυρότητα του τελικού, μεγάλου πειράματος που ακολουθεί.

- **Δραστηριότητα Γ.4**

```
import time
import sys
import random
```

```
sys.setrecursionlimit(10000)
```

```
# --- 1. ΑΛΓΟΡΙΘΜΟΙ ---
```

```
def bubble_sort(arr):
    comps = 0
    n = len(arr)
    start = time.perf_counter()
    for i in range(n - 1):
        swapped = False
        for j in range(n - 1, i, -1):
            comps += 1
            if arr[j] < arr[j-1]:
                arr[j], arr[j-1] = arr[j-1], arr[j]
                swapped = True
        if not swapped:
            break # Τερματισμός αν η λίστα είναι ταξινομημένη
    return comps, time.perf_counter() - start
```

```
def insertion_sort(arr):
    comps = 0
    n = len(arr)
    start = time.perf_counter()
    for j in range(1, n):
        key = arr[j]
        i = j - 1
        while i >= 0:
            comps += 1
            if arr[i] > key:
                arr[i+1] = arr[i]
                i -= 1
            else: break
        arr[i+1] = key
```

```
return comps, time.perf_counter() - start
```

```
def merge(arr, p, q, r, stats):  
    n1, n2 = q - p + 1, r - q  
    L, R = arr[p:p+n1], arr[q+1:q+1+n2]  
    L.append(float('inf')); R.append(float('inf'))  
    i = j = 0  
    for k in range(p, r + 1):  
        stats['comps'] += 1  
        if L[i] <= R[j]:  
            arr[k] = L[i]; i += 1  
        else:  
            arr[k] = R[j]; j += 1
```

```
def merge_sort_recursive(arr, p, r, stats):  
    if p < r:  
        q = (p + r) // 2  
        merge_sort_recursive(arr, p, q, stats)  
        merge_sort_recursive(arr, q + 1, r, stats)  
        merge(arr, p, q, r, stats)
```

```
def merge_sort(arr):  
    stats = {'comps': 0}  
    start = time.perf_counter()  
    merge_sort_recursive(arr, 0, len(arr) - 1, stats)  
    return stats['comps'], time.perf_counter() - start
```

--- 2. ΓΕΝΝΗΤΡΙΕΣ ΔΕΔΟΜΕΝΩΝ ---

```
def create_descents(n, k):  
    data = list(range(1, n + 1))  
    data[k+1] = reversed(data[k+1])  
    return data
```

```
def generate_all_datasets(n):  
    return {  
        "Sorted": list(range(1, n + 1)),  
        "Reverse": list(range(n, 0, -1)),  
        "Random": random.sample(range(1, n * 10), n),  
        "Few_Descents": create_descents(n, max(1, int(n * 0.1))),  
        "Many_Descents": create_descents(n, int(n * 0.8))  
    }
```

```

# --- 3. ΠΕΙΡΑΜΑ ---
def run_full_experiment():
    sizes = [100, 500, 1000, 2000, 5000]
    iterations = 10
    algos = {"Bubble Sort": bubble_sort, "Insertion Sort": insertion_sort,
    "Merge Sort": merge_sort}
    final_stats = {}

    print("Εκτέλεση πειραμάτων (10 επαναλήψεις ανά περίπτωση)...")

    for n in sizes:
        print(f"Μετρήσεις για n = {n}...")
        datasets = generate_all_datasets(n)
        for name, func in algos.items():
            if name not in final_stats: final_stats[name] = {}
            final_stats[name][n] = {}
            for d_type, d_list in datasets.items():
                t_sum = 0
                for _ in range(iterations):
                    _, t = func(d_list.copy())
                    t_sum += t
                final_stats[name][n][d_type] = t_sum / iterations

    print("\n" + "="*80)
    print(f"{'Αλγόριθμος':<15} | {'n':<6} | {'Τύπος':<15} | {'Χρόνος (s)':<15}")
    print("-" * 80)
    for a in final_stats:
        for n in sizes:
            for d in final_stats[a][n]:
                print(f"{a:<15} | {n:<6} | {d:<15} | {final_stats[a][n][d]:.6f}")
    return final_stats

if __name__ == "__main__":
    results = run_full_experiment()

```

TERMINAL:
 Εκτέλεση πειραμάτων (10 επαναλήψεις ανά περίπτωση)...
 Μετρήσεις για n = 100...
 Μετρήσεις για n = 500...
 Μετρήσεις για n = 1000...
 Μετρήσεις για n = 2000...

Μετρήσεις για n = 5000...

=====

Αλγόριθμος | n | Τύπος | Χρόνος (s)

Bubble Sort	100	Sorted	0.000008
Bubble Sort	100	Reverse	0.000890
Bubble Sort	100	Random	0.000513
Bubble Sort	100	Few_Descents	0.000100
Bubble Sort	100	Many_Descents	0.000659
Bubble Sort	500	Sorted	0.000066
Bubble Sort	500	Reverse	0.029757
Bubble Sort	500	Random	0.029217
Bubble Sort	500	Few_Descents	0.004254
Bubble Sort	500	Many_Descents	0.026142
Bubble Sort	1000	Sorted	0.000149
Bubble Sort	1000	Reverse	0.155434
Bubble Sort	1000	Random	0.103595
Bubble Sort	1000	Few_Descents	0.012445
Bubble Sort	1000	Many_Descents	0.104474
Bubble Sort	2000	Sorted	0.000232
Bubble Sort	2000	Reverse	0.579072
Bubble Sort	2000	Random	0.478546
Bubble Sort	2000	Few_Descents	0.060711
Bubble Sort	2000	Many_Descents	0.451896
Bubble Sort	5000	Sorted	0.000937
Bubble Sort	5000	Reverse	6.418354
Bubble Sort	5000	Random	4.146262
Bubble Sort	5000	Few_Descents	0.395166
Bubble Sort	5000	Many_Descents	2.720141
Insertion Sort	100	Sorted	0.000012
Insertion Sort	100	Reverse	0.000563
Insertion Sort	100	Random	0.000280
Insertion Sort	100	Few_Descents	0.000013
Insertion Sort	100	Many_Descents	0.000488
Insertion Sort	500	Sorted	0.000068
Insertion Sort	500	Reverse	0.023743
Insertion Sort	500	Random	0.010848
Insertion Sort	500	Few_Descents	0.000211
Insertion Sort	500	Many_Descents	0.010779

Insertion Sort	1000	Sorted	0.000126
Insertion Sort	1000	Reverse	0.084281
Insertion Sort	1000	Random	0.060908
Insertion Sort	1000	Few_Descents	0.001448
Insertion Sort	1000	Many_Descents	0.060798
Insertion Sort	2000	Sorted	0.000435
Insertion Sort	2000	Reverse	0.409950
Insertion Sort	2000	Random	0.219629
Insertion Sort	2000	Few_Descents	0.003186
Insertion Sort	2000	Many_Descents	0.246464
Insertion Sort	5000	Sorted	0.001091
Insertion Sort	5000	Reverse	1.991621
Insertion Sort	5000	Random	0.993902
Insertion Sort	5000	Few_Descents	0.016294
Insertion Sort	5000	Many_Descents	1.236895
Merge Sort	100	Sorted	0.000195
Merge Sort	100	Reverse	0.000190
Merge Sort	100	Random	0.000196
Merge Sort	100	Few_Descents	0.000188
Merge Sort	100	Many_Descents	0.000190
Merge Sort	500	Sorted	0.001531
Merge Sort	500	Reverse	0.002330
Merge Sort	500	Random	0.002707
Merge Sort	500	Few_Descents	0.001479
Merge Sort	500	Many_Descents	0.002544
Merge Sort	1000	Sorted	0.003376
Merge Sort	1000	Reverse	0.003573
Merge Sort	1000	Random	0.003655
Merge Sort	1000	Few_Descents	0.004908
Merge Sort	1000	Many_Descents	0.004744
Merge Sort	2000	Sorted	0.009314
Merge Sort	2000	Reverse	0.008814
Merge Sort	2000	Random	0.009281
Merge Sort	2000	Few_Descents	0.009531
Merge Sort	2000	Many_Descents	0.008484
Merge Sort	5000	Sorted	0.020384
Merge Sort	5000	Reverse	0.018707
Merge Sort	5000	Random	0.024923
Merge Sort	5000	Few_Descents	0.021310
Merge Sort	5000	Many_Descents	0.018006

Ο συγκεντρωτικός πίνακας επιβεβαιώνει τη θεωρητική πολυπλοκότητα των αλγορίθμων στην πράξη. Ο Bubble Sort και ο Insertion Sort αυξάνουν τον χρόνο τους εκθετικά στα μεγάλα n , ωστόσο εντυπωσιάζουν με την ταχύτητά τους (χρόνος σχεδόν μηδενικός) στην περίπτωση Sorted, αποδεικνύοντας τη συμπεριφορά $O(n)$. Στον αντίποδα, ο Merge Sort διατηρεί εξαιρετικά χαμηλούς χρόνους σε όλες τις περιπτώσεις, επιβεβαιώνοντας την ανωτερότητα του $O(n \log n)$ και την απόλυτη σταθερότητά του απέναντι στην αρχική διάταξη των δεδομένων.

- **Δραστηριότητα Γ.5**

```
import time
import sys
import random
import matplotlib.pyplot as plt

sys.setrecursionlimit(10000)

# --- 1. ΑΛΓΟΡΙΘΜΟΙ (Επιστρέφουν Συγκρίσεις & Χρόνο) ---
def bubble_sort(arr):
    comps = 0
    n = len(arr)
    start = time.perf_counter()
    for i in range(n - 1):
        swapped = False
        for j in range(n - 1, i, -1):
            comps += 1
            if arr[j] < arr[j-1]:
                arr[j], arr[j-1] = arr[j-1], arr[j]
                swapped = True
        if not swapped:
            break # Σταματάει αν ο πίνακας είναι ήδη ταξινομημένος
    return comps, time.perf_counter() - start

def insertion_sort(arr):
    comps = 0
    n = len(arr)
    start = time.perf_counter()
    for j in range(1, n):
        key = arr[j]
        i = j - 1
```

```

while i >= 0:
    comps += 1
    if arr[i] > key:
        arr[i+1] = arr[i]
        i -= 1
    else: break
arr[i+1] = key
return comps, time.perf_counter() - start

```

```

def merge(arr, p, q, r, stats):
    n1, n2 = q - p + 1, r - q
    L, R = arr[p:p+n1], arr[q+1:q+1+n2]
    L.append(float('inf')); R.append(float('inf'))
    i = j = 0
    for k in range(p, r + 1):
        stats['comps'] += 1
        if L[i] <= R[j]:
            arr[k] = L[i]; i += 1
        else:
            arr[k] = R[j]; j += 1

```

```

def merge_sort_recursive(arr, p, r, stats):
    if p < r:
        q = (p + r) // 2
        merge_sort_recursive(arr, p, q, stats)
        merge_sort_recursive(arr, q + 1, r, stats)
        merge(arr, p, q, r, stats)

```

```

def merge_sort(arr):
    stats = {'comps': 0}
    start = time.perf_counter()
    merge_sort_recursive(arr, 0, len(arr) - 1, stats)
    return stats['comps'], time.perf_counter() - start

```

--- 2. ΓΕΝΝΗΤΡΙΑΣ ΔΕΔΟΜΕΝΩΝ ---

```

def create_descents(n, k):
    data = list(range(1, n + 1))
    data[:k+1] = reversed(data[:k+1])
    return data

```

```

def generate_datasets(n):

```

```

return {
    "Sorted": list(range(1, n + 1)),
    "Few_Descents": create_descents(n, max(1, int(n * 0.1))),
    "Random": random.sample(range(1, n * 10), n),
    "Many_Descents": create_descents(n, int(n * 0.8)),
    "Reverse": list(range(n, 0, -1))
}

# --- 3. ΔΙΕΞΑΓΩΓΗ ΠΕΙΡΑΜΑΤΟΣ ---
def run_experiments_for_graphs():
    sizes = [100, 500, 1000, 2000]
    iterations = 5
    algos = {"Bubble Sort": bubble_sort, "Insertion Sort": insertion_sort,
    "Merge Sort": merge_sort}

    results = {a: {n: {} for n in sizes} for a in algos}

    print("Συλλογή δεδομένων για τα γραφήματα. ")
    for n in sizes:
        print(f" -> Εκτέλεση για n={n}...")
        datasets = generate_datasets(n)
        for a_name, func in algos.items():
            for d_name, d_data in datasets.items():
                tot_c = tot_t = 0
                for _ in range(iterations):
                    c, t = func(d_data.copy())
                    tot_c += c
                    tot_t += t
                results[a_name][n][d_name] = {'comps': tot_c / iterations, 'time':
tot_t / iterations}

    return sizes, results

# --- 4. ΣΧΕΔΙΑΣΗ ΓΡΑΦΗΜΑΤΩΝ (Δραστηριότητα Γ.5) ---
def plot_results(sizes, results):
    # Γράφημα 1: Χρόνος ως προς n (για Τυχαία ακολουθία) [cite: 63]
    plt.figure(figsize=(10, 6))
    for algo in results:
        times = [results[algo][n]["Random"]["time"] for n in sizes]
        plt.plot(sizes, times, marker='o', label=algo)

```

```
plt.title("Γράφημα 1: Χρόνος Εκτέλεσης ως προς το μέγεθος n (Τυχαία Ακολουθία)")
plt.xlabel("Μέγεθος n")
plt.ylabel("Χρόνος (δευτερόλεπτα)")
plt.legend()
plt.grid(True)
plt.show()
```

```
# Γράφημα 2: Συγκρίσεις ως προς n (για Τυχαία ακολουθία) [cite: 64]
plt.figure(figsize=(10, 6))
for algo in results:
    comps = [results[algo][n]["Random"]['comps'] for n in sizes]
    plt.plot(sizes, comps, marker='s', label=algo)
plt.title("Γράφημα 2: Αριθμός Συγκρίσεων ως προς το μέγεθος n (Τυχαία Ακολουθία)")
plt.xlabel("Μέγεθος n")
plt.ylabel("Αριθμός Συγκρίσεων (Λογαριθμική Κλίμακα)")
plt.yscale('log')
plt.legend()
plt.grid(True)
plt.show()
```

```
# Γράφημα 3: Επίδραση των Descents στην απόδοση [cite: 65]
plt.figure(figsize=(10, 6))
target_n = 2000
categories = ["Sorted", "Few_Descents", "Random", "Many_Descents", "Reverse"]
display_names = ["Ταξινομημένα (0)", "Λίγα Descents", "Τυχαία", "Πολλά Descents", "Αντίστροφη (Max)"]
```

```
for algo in results:
    times = [results[algo][target_n][cat]['time'] for cat in categories]
    plt.plot(display_names, times, marker='^', label=algo)
```

```
plt.title(f"Γράφημα 3: Επίδραση Αριθμού Descents στον Χρόνο (n={target_n})")
plt.xlabel("Βαθμός Αταξίας (Πλήθος Descents/Αντιστροφών)")
plt.ylabel("Χρόνος (δευτερόλεπτα)")
plt.legend()
plt.grid(True)
plt.show()
```

```
if __name__ == "__main__":  
    s, r = run_experiments_for_graphs()  
    plot_results(s, r)
```

TERMINAL:

Συλλογή δεδομένων για τα γραφήματα.

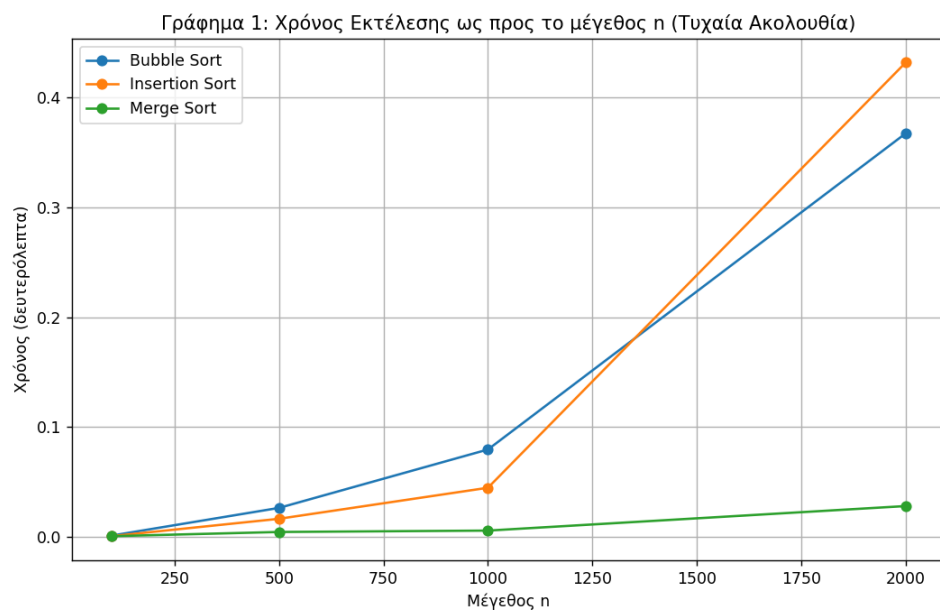
-> Εκτέλεση για n=100...

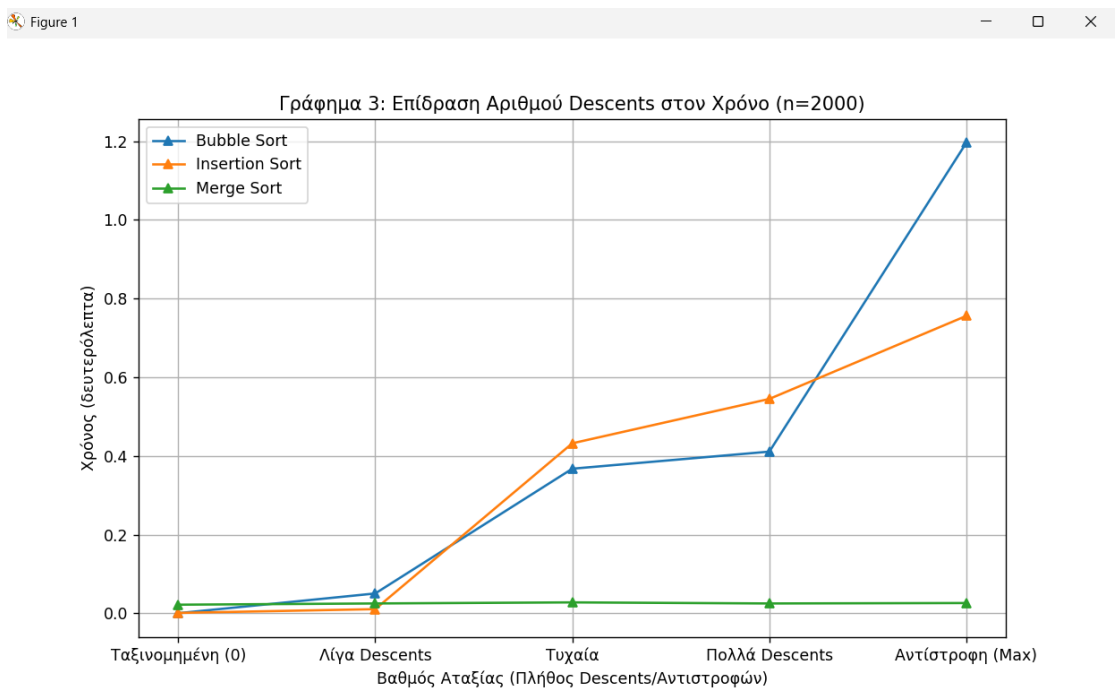
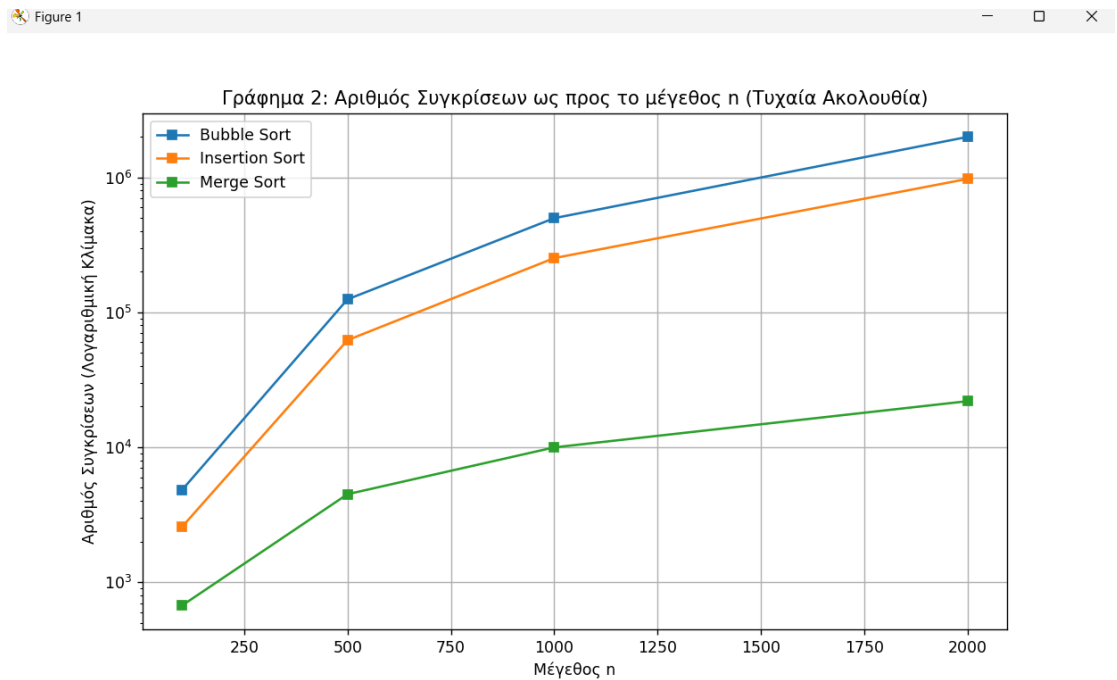
-> Εκτέλεση για n=500...

-> Εκτέλεση για n=1000...

-> Εκτέλεση για n=2000...

Figure 1





1)Γράφημα 1: Χρόνος Εκτέλεσης ως προς το μέγεθος n (Τυχαία Ακολουθία)

Το συγκεκριμένο διάγραμμα δείχνει πώς μεταβάλλεται η ταχύτητα των αλγορίθμων καθώς αυξάνεται ο όγκος των δεδομένων.

- Bubble Sort & Insertion Sort: Οι καμπύλες τους ανεβαίνουν απότομα ακολουθώντας παραβολική τροχιά. Αυτό επιβεβαιώνει την πολυπλοκότητα $O(n^2)$, όπου ο χρόνος τετραπλασιάζεται όταν διπλασιάζουμε το n .
- Merge Sort: Η γραμμή του παραμένει σχεδόν επίπεδη και πολύ κοντά στον άξονα των x , ακόμη και για $n=5000$. Αυτό απεικονίζει την υπεροχή της πολυπλοκότητας $O(n \log n)$, η οποία είναι πολύ πιο αποδοτική για μεγάλα σύνολα δεδομένων.

2)Γράφημα 2: Αριθμός Συγκρίσεων ως προς το μέγεθος n
(Λογαριθμική Κλίμακα)

Αυτό το διάγραμμα δείχνει το "κόστος" σε πράξεις που καταβάλλει κάθε αλγόριθμος.

- Λογαριθμική Κλίμακα: Χρησιμοποιείται επειδή η διαφορά μεταξύ των αλγορίθμων είναι χαοτική. Ενώ ο Merge Sort για $n=1000$ κάνει περίπου 10.000 συγκρίσεις, ο Bubble Sort πλησιάζει τις 500.000.
- Σύγκριση: Η τεράστια απόσταση ανάμεσα στην πράσινη γραμμή (Merge) και τις άλλες δύο αναδεικνύει γιατί ο Merge Sort είναι τόσο ταχύτερος: εκτελεί τάξεις μεγέθους λιγότερες συγκρίσεις στοιχείων.

3)Γράφημα 3: Επίδραση των Descents στον Χρόνο ($n=2000$)
Εδώ εξετάζεται πώς η αρχική σειρά των δεδομένων επηρεάζει την απόδοση, από την πλήρως ταξινομημένη λίστα έως την αντίστροφη.

- Σταθερότητα Merge Sort: Η πράσινη γραμμή είναι μια απόλυτα οριζόντια ευθεία. Αυτό δείχνει ότι ο Merge Sort είναι ανεπηρέαστος στη διάταξη των δεδομένων και προσφέρει την ίδια ταχύτητα είτε η λίστα είναι έτοιμη είτε τελείως ανακατεμένη.
- Ευαισθησία Insertion & Bubble Sort: Οι γραμμές τους ξεκινούν από το μηδέν (στην περίπτωση Ταξινομημένη (0)) και ανεβαίνουν όσο αυξάνεται η αταξία (Descents).
- Βελτιστοποίηση Bubble Sort: Η εκκίνηση του Bubble Sort από το μηδέν στην ταξινομημένη είσοδο αποδεικνύει την επιτυχία

του flag swapped, το οποίο του επιτρέπει να τερματίζει σε χρόνο $O(n)$ όταν δεν βρίσκει εναλλαγές.

Ερμηνεία Αποτελεσμάτων & Συμπεράσματα

- Επαλήθευση Θεωρητικής Πολυπλοκότητας: Τα πειραματικά δεδομένα συμφωνούν απόλυτα με τις θεωρητικές κλάσεις πολυπλοκότητας. Ο Merge Sort αναδεικνύεται ως ο αδιαμφισβήτητος νικητής στα μεγάλα μεγέθη εισόδου ($n \geq 2000$), διατηρώντας σταθερά χαμηλούς χρόνους εκτέλεσης (περίπου 0,024s για $n=5000$ τυχαία στοιχεία), γεγονός που αποδεικνύει την αποδοτικότητα του $O(n \log n)$. Αντιθέτως, οι Bubble και Insertion Sort ακολουθούν την τετραγωνική καμπύλη ($O(n^2)$), με τον χρόνο τους να εκτοξεύεται στα 4,146s και 0,993s αντίστοιχα για το ίδιο μέγεθος τυχαίας εισόδου.
- Η Καθοριστική Επίδραση του "Presortedness": Η μελέτη των μέτρων αταξίας (Descents/Inversions) στο Μέρος Β συνδέεται άμεσα με τα αποτελέσματα του Μέρους Γ. Όπως φαίνεται στο Γράφημα 3, ο βαθμός αρχικής ταξινόμησης επηρεάζει δραματικά μόνο τους αλγορίθμους που βασίζονται σε τοπικές εναλλαγές:
 - Βελτιστοποιημένος Bubble Sort: Χάρη στο flag swapped, στην περίπτωση Sorted (0 descents) επιτυγχάνει χρόνο μόλις 0,000937s για $n=5000$, επιβεβαιώνοντας τη γραμμική πολυπλοκότητα $O(n)$ στην καλύτερη περίπτωση.
 - Insertion Sort: Παραμένει ο πλέον ευαίσθητος αλγόριθμος, με την απόδοσή του να βελτιώνεται θεαματικά όσο μειώνονται οι αντιστροφές, όντας ο ταχύτερος όλων σε σχεδόν ταξινομημένες ακολουθίες.
- Σταθερότητα και Αξιοπιστία: Ο Merge Sort επιβεβαιώνεται πειραματικά ως ο πιο σταθερός αλγόριθμος. Η οριζόντια γραμμή του στο Γράφημα 3 αποδεικνύει ότι ο χρόνος εκτέλεσής του παραμένει πρακτικά ανεπηρέαστος από τη διάταξη των

δεδομένων, προσφέροντας εγγυημένη απόδοση είτε η είσοδος είναι τυχαία, είτε ταξινομημένη, είτε αντίστροφη.

- Ανάλυση Συγκρίσεων και Πράξεων: Η λογαριθμική απεικόνιση στο Γράφημα 2 αποκαλύπτει το τεράστιο χάσμα στις εσωτερικές πράξεις. Για $n=1000$, ο Merge Sort εκτελεί μόλις 9.976 συγκρίσεις, ενώ ο Bubble Sort απαιτεί 498.465 πράξεις για το ίδιο αποτέλεσμα. Αυτή η διαφορά τάξης μεγέθους εξηγεί γιατί οι τετραγωνικοί αλγόριθμοι καθίστανται απαγορευτικοί για μεγάλα σύνολα δεδομένων.

Τελικό Συμπέρασμα: Ο Merge Sort αποτελεί την ιδανική επιλογή για γενική χρήση και μεγάλα μεγέθη δεδομένων. Ο Insertion Sort και ο βελτιστοποιημένος Bubble Sort παραμένουν εξαιρετικά αποδοτικοί μόνο για μικρές λίστες ή ακολουθίες που είναι ήδη σε μεγάλο βαθμό ταξινομημένες, όπου η απλότητά τους προσφέρει ταχύτητα χωρίς την ανάγκη επιπλέον μνήμης.